

AWS CloudTrail Flaws2 Investigation

Step-by-Step Guide for All 6 Objectives

Objective 1 — Download CloudTrail Logs

What you're doing:

Configure AWS CLI and download CloudTrail logs from the public **flaws2-logs** S3 bucket for offline investigation.

Commands:

```
aws sts get-caller-identity
aws s3 sync s3://flaws2-logs .
Get-ChildItem
```

Screenshots to capture:

- AWS identity confirmation output
- S3 sync progress/completion
- Downloaded AWSLogs folder listing

Objective 2 — Access Target Account Using AssumeRole

What you're doing:

Use AWS CLI profile **target_security** to assume a cross-account role in the Target account and list its S3 buckets.

- Account **322079859186** = Security account (IAM user)
- Account **653711331788** = Target account (assumed role)

~/.aws/config profile setup:

```
[default]
region=us-east-1
output=json

[profile target_security]
region=us-east-1
output=json
source_profile = default
role_arn = arn:aws:iam::653711331788:role/security
```

Commands:

```
aws sts get-caller-identity
aws --profile target_security sts get-caller-identity
aws --profile target_security s3 ls
```

Screenshots to capture:

- Default identity
- target_security assumed role identity
- Target account S3 bucket listing

Objective 3 — Analyze CloudTrail Logs with jq

What you're doing:

Decompress .gz log files, then use **jq** to parse and filter JSON events into a timeline and export to CSV.

Step 1 — Decompress all .gz files (PowerShell):

```
Get-ChildItem -Recurse *.gz | ForEach-Object {
    $inputFile = $_.FullName
    $outputFile = $inputFile -replace '\.gz$', ''
    $input = [System.IO.File]::OpenRead($inputFile)
    $output = [System.IO.File]::Create($outputFile)
    $gzip = New-Object System.IO.Compression.GzipStream(
        $input,
        [System.IO.Compression.CompressionMode]::Decompress)
    $gzip.CopyTo($output)
    $gzip.Close(); $input.Close(); $output.Close()
}
```

Step 2 — Quick timeline in GridView:

```
Get-ChildItem -Filter *.json | Get-Content |
jq -r '.Records[] |
    [.eventTime, .sourceIPAddress,
     .userIdentity.arn, .userIdentity.accountId,
     .userIdentity.type, .eventName] | @csv' |
ConvertFrom-Csv -Header Time,SourceIP,ARN,AccountId,IdentityType,EventName |
Sort-Object Time |
Out-GridView
```

Step 3 — Full export to CSV:

```
Get-ChildItem -Filter *.json | Get-Content |
jq -r '.Records[] |
    [.eventTime, .eventSource, .eventName,
     .sourceIPAddress, .userIdentity.type,
     .userIdentity.arn, .userIdentity.accountId,
     (.requestParameters | tostring),
     (.responseElements | tostring),
     .errorCode, .errorMessage, .userAgent] | @csv' |
ConvertFrom-Csv -Header Time,EventSource,EventName,SourceIP,
    IdentityType,ARN,AccountId,RequestParams,ResponseElements,
    ErrorCode,ErrorMessage,UserAgent |
Sort-Object Time |
Export-Csv .\evidence\cloudtrail_full_view.csv -NoTypeInformation
```

Screenshots to capture:

- Decompressed JSON files listing
- jq timeline GridView output
- CSV file created in evidence folder

Objective 4 — Identify Credential Theft

What you're doing:

Find the **ListBuckets** event in logs to identify which role performed it, then inspect the **level3** role trust policy to understand how it was assumed.

Commands:

```
# Find and display ListBuckets events
Get-ChildItem -Filter *.json | Get-Content |
```

```
jq '.Records[] | select(.eventName == "ListBuckets")'

# Save ListBuckets event as evidence
Get-ChildItem -Filter *.json | Get-Content |
jq '.Records[] | select(.eventName == "ListBuckets")' > .\evidence\listbuckets_event.json

# Inspect the level3 IAM role
aws --profile target_security iam get-role --role-name level3

# Extract only the trust policy document
aws --profile target_security iam get-role --role-name level3 |
jq '.Role.AssumeRolePolicyDocument'
```

Screenshots to capture:

- ListBuckets event JSON output
- level3 role details
- Trust policy showing who can assume the role

Objective 5 — IAM / ECR Policy Analysis

What you're doing:

Retrieve and inspect the ECR repository policy for **level2** to confirm it allows **Principal "*" (anyone)** to perform read actions: *ListImages*, *BatchGetImage*, *GetDownloadUrlForLayer*.

Commands:

```
# Get raw ECR repository policy
aws --profile target_security ecr get-repository-policy --repository-name level2

# Pretty-print the policy JSON
aws --profile target_security ecr get-repository-policy --repository-name level2 |
jq '.policyText | fromjson'

# Save cleaned policy as evidence
aws --profile target_security ecr get-repository-policy --repository-name level2 |
jq '.policyText | fromjson' > .\evidence\ecr_policy_level2.json
```

Screenshots to capture:

- Raw ECR policy output
- Cleaned JSON showing Principal "*"
- ecr_policy_level2.json saved to evidence folder

Objective 6 — Athena SQL Investigation

What you're doing:

Use Amazon Athena to query CloudTrail logs directly from S3 with SQL — no download required.

Step 1 — Create database:

```
CREATE DATABASE flaws2;
```

Step 2 — Create external table:

```
CREATE EXTERNAL TABLE `cloudtrail`(
  `eventversion`      string,
  `useridentity`      struct<type:string, principalid:string,
                        arn:string, accountid:string,
```

```

        invokedby:string, accesskeyid:string,
        sessioncontext:struct<
            attributes:struct<
                mfaauthenticated:string,
                creationdate:string>,
            sessionissuer:struct<
                type:string, principalid:string,
                arn:string, accountid:string,
                username:string>>>,
        `eventtime`          string,
        `eventsources`       string,
        `eventname`          string,
        `awsregion`          string,
        `sourceipaddress`    string,
        `useragent`          string,
        `errorcode`          string,
        `errormessage`       string,
        `requestparameters`  string,
        `responseelements`   string,
        `additionaleventdata` string,
        `requestid`          string,
        `eventid`            string,
        `resources`          array<struct<arn:string,
                                accountid:string, type:string>>,
        `eventtype`          string,
        `apiversion`         string,
        `readonly`           string,
        `recipientaccountid` string,
        `serviceeventdetails` string,
        `sharedeventid`      string,
        `vpceendpointid`     string
    )
ROW FORMAT SERDE
    'com.amazon.emr.hive.serde.CloudTrailSerde'
STORED AS INPUTFORMAT
    'com.amazon.emr.cloudtrail.CloudTrailInputFormat'
OUTPUTFORMAT
    'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION
    's3://flaws2-logs/AWSLogs/653711331788/CloudTrail';

```

Step 3 — Query: Full event timeline:

```

SELECT eventtime, eventname
FROM cloudtrail
ORDER BY eventtime;

```

Step 4 — Query: Event counts (most frequent first):

```

SELECT eventname, count(*) AS mycount
FROM cloudtrail
GROUP BY eventname
ORDER BY mycount DESC;

```

Step 5 — Query: All ListBuckets events:

```

SELECT eventtime, eventname, sourceipaddress,
       useridentity.type, useridentity.arn, useragent
FROM cloudtrail

```

```
WHERE eventname = 'ListBuckets';
```

Step 6 — Query: All ECR-related events:

```
SELECT eventtime, eventname, sourceipaddress,  
       useridentity.type, useridentity.arn,  
       requestparameters, useragent  
FROM cloudtrail  
WHERE eventsource = 'ecr.amazonaws.com'  
ORDER BY eventtime;
```

Screenshots to capture:

- Athena database flaws2 created
- cloudtrail table created
- Event timeline query results
- Event count query results
- ListBuckets query results

Cleanup: DROP TABLE cloudtrail; | DROP DATABASE flaws2; | Delete Athena result S3 bucket